System Architecture & Deployment Topology/Arrangement

The AEGIS system uses a Distributed Cloud Architecture by exploiting production-ready free-tier platforms. This architecture isolates the presentation and application orchestration layers as well as the data persistence and untrusted code execution code execution sandbox.

Component Selection & Comparative Analysis

Frontend Hosting: Vercel

- **Approach**: Serverless deployment by using Vercel's Edge Network.
- **Alternatives Considered:** Self-hosting on a traditional Linux server, AWS S3 and CloudFront.
  - **Justification vs. Alternatives:** AWS S3 and CloudFront offer enterprise scaling; their configuration carries variable data-egress financial risks. Vercel serves static web assets through a global Content Delivery Network (CDN) straight out of the box without any cost. This guarantees fast initial page loads globally.
- **Non-Functional Requirement Alignment: Scalability & Concurrency:** The CDN design can handle traffic spikes up to and beyond 500 concurrent users without UI latency.
  - **Maintainability:** Inherit GitHub integration triggers automatic compilation and deployment upon code merges, meeting the required **2-hour deployment window requirement.**

Commented [1]: (a globally distributed CDN and compute platform that serves your web content and executes code from Points of Presence (PoPs) closest to your users, drastically reducing load times)

Backend Orchestration Layer: FastAPI on Koyeb

- **Approach:** Containerised deployment on Koyeb's Free Tier (Micro Instance).
- **Alternatives Considered:** Render (Free Tier), Railway (Paid).
  - **Justification vs. Alternatives:** Traditional free tiers (like Render) put backend applications in a "sleep" state after 15 minutes of inactivity, which causes a 30-to-50 second "cold start" delay. Koyeb's free tier provides an instance that remains active continuously, which eliminates the cold starts.

Commented [2]: a type of small, affordable cloud server (like an Amazon EC2 instance) designed for low-traffic websites, testing environments, and lightweight background tasks

- **Non-Functional Requirement Alignment:**
    - **Performance:** Keeping the backend instance awake ensures that API routing begins processing immediately, which is required for achieving a **< 2-second response time.**
        - **Data Persistence Layer: Supabase (Managed PostgreSQL)**
- **Approach:** Cloud-managed PostgreSQL instance by Supabase.
- **Alternatives Considered:** Self-hosted PostgreSQL via Docker, SQLite.
    - **Justification vs. Alternatives:** SQLite does not have the required concurrent write capabilities. Self-hosting PostgreSQL on the free tier heavily consumes memory, risking Out-Of-Memory (OOM) crashes. Supabase provides an isolated database environment at no cost.
- **Non-Functional Requirement Alignment:**
    - **Security:** Natively, Supabase manages **AES-256 encryption at rest and SSL encryption in transit**, satisfying data privacy requirements.
    - **Scalability:** Includes Supervisor for database connection pooling, preventing connection exhaustion when 500 concurrent users execute database queries.

Commented [3]: Supabase automatically handles securing your data across its entire lifecycle without requiring any setup or configuration on your part.

Commented [4]: Supabase has a built-in traffic cop (called Supavisor) that stops your database from crashing when hundreds of users use your app at the exact same time

Code Execution Sandbox: Piston

- **Approach:** A containerized, self-hosted Piston Engine running inside Docker on an Oracle Cloud Ampere A1 Compute Instance.
- **Alternatives Considered:** Judge0 Community Edition, Hosted Piston API (Public), Raw Server Execution.
    - **Justification vs. Alternatives:** Piston was chosen over Judge0 because Judge0 is very resource-intensive; it requires multiple secondary containers such as a Redis queue, a dedicated PostgreSQL database instance, and background workers to complete a single code run. Piston operates as a **single, stateless, high-performance API container**. By removing the memory and CPU overhead of Redis and an extra database, Piston leaves maximum system memory and CPU cycles available strictly for compiling user submissions.
        - **Why Self-Hosted over Public API:** Public-hosted Piston Api enforces strict rate limits that would instantly cause dropped requests during stress testing. Self-hosting on Oracle's free ARM server (4 vCPUs, 24GB RAM) helps to remove request caps.
    - **Non-Functional Requirement Alignment:**

Commented [LN5]: Data structure that uses FIFO to handle jobs

- **Performance:** Piston's stateless execution pipeline lowers internal engine processing latency, giving the system the best possible speed to execute code and **return payloads under the 2-second limit.**
- **Concurrency:** Piston has a small idle memory footprint compared to Judge0; the server can comfortably spin up and tear down isolated code execution environments for multiple language runtimes without bottlenecking.
- **Security:** Piston executes untrusted code in secure, short-term environments with restricted network privileges. Malicious code submissions are containerized and cannot breach the parent server.

Commented [LN6]: An **idle memory footprint** is the amount of RAM a software application, operating system, or container consumes when it is running but not actively performing tasks or processing user data.

Non-Functional Requirements Traceability Matrix

NFR Identifier	Quality Requirement	Architectural Decision	Evaluation Status
Performance: NFR-01	Response time <= 2 seconds	Asynchronous Webhook Architecture + Stateless, low-overhead Piston Engine API eliminates internal engine bottlenecks.	N/A
Concurrency: NFR-02	Support for 500+ concurrent users	Vercel Global Edge CDN handles frontend volume; Piston's ultra-low memory footprint prevents server crashes.	N/A
Security: NFR-03	Secure execution of untrusted code	Dockerized Piston execution sandboxes isolate user code. Supabase handles AEGIS-256 data encryption.	N/A
Reliability: NFR-04	Availability >= 99.5% uptime	Distributed architecture limits single points of failure. Koyeb and Vercel maintain native uptime monitoring.	N/A
Maintainability: NFR-05	Automated build verification and deployment < 2 hours	Automated Git-driven CI/CD integration natively linked to Vercel and Koyeb platform pipelines.	N/A

Commented [LN7]: Need to revisit to formulate an execution plan.

Operational Deployment Specifications

Environment Parity Configuration

The system isolates operational contexts using Git branch tracking:

- **Development Environment:** Configured locally via the team's individual workstations by using Docker Compose containers pointing to a separate Supabase local/dev schema.
- Production Environment: Triggered automatically via GitHub Webhooks. Any pull requests merged into the main branch initiate an immediate secure cloud compilation and update the live endpoints on Vercel and Koyeb with no manual intervention.

Infrastructure as Code & Reproducibility

The deployment ecosystem is documented and defined via a root-level **docker-compose.yml** file to prevent configuration drift and satisfy reproducibility.
Since Piston does not make use of additional tracking databases, the configuration remains simple, allowing any evaluator to clone the repository and run a local mirror with a single command: **docker compose up –build**.

Secrets Management Protocol

Credentials, connection strings, and cryptographic keys are strictly excluded from version control tracking:

- An explicit template file **(.env.example)** is maintained publicly in the root directory to specify structural dependencies.
- The production **.env** configuration file is directly attached to the system's **.gitignore** rules.
- Production keys are injected into application memory at runtime through Vercel and Koyeb's secure Web UI Environment Panels, ensuring that no plain-text storage exists in code repositories.

Automated Rollback Engineering

In the event of an unexpected failure or unstable deployment during runtime evaluations, a repeatable rollback strategy is utilized:

- **Edge Layer (Frontend - Vercel):** Vercel's Dashboard retains immutable historical build images tied directly to their respective Git commit hashes. A rollback is executed instantly by designating a previous stable SHA as the active production target (recovery time: < 10 seconds). Thus, our system utilizes Vercel's native **Image Tag Pinning.**

- **Application Layer (Backend - Koyeb):** Koyeb's native deployment history is leveraged. Rollbacks are executed via the Koyeb API/CLI by instantly rerouting traffic back to the cached, last-known-stable container image artifact (recovery time < 30 seconds).
  Thus, our system utilizes Koyeb's native **Immutable Artifact Rollback**.
- **Execution Layer (Piston – Self-Hosted Oracle):** The Piston engine operates under an immutable container lifecycle model, keeping active and standby instances running simultaneously on separate ports. Rollbacks are executed instantly via a lightweight reverse proxy that reroutes traffic back to the stable container port (recovery time < 5 seconds). Thus, our system utilizes **Blue-Green Port Switching**.

Architectural Evolution & Migration Path

While the selected distributed free-tier setup allows the project to be delivered by utilizing free-tier options, the team acknowledges that running cross-cloud HTTP communications (Koyeb - Oracle Cloud - Supabase) introduces slight latency.

As a defined roadmap extension, moving the application logic and the Piston engine container onto a single **Dedicated Virtual Private Server (VPS)** via platforms like DigitalOcean, Linode, or Hetzner (R80 – R170) will consolidate operations onto localhost. This future phase will remove inter-network routing latency, securely isolate scheduling, and effortlessly scale up processing power to allow heavy concurrent compiler volumes.

Rejection of Industry-Standard Hyper-Scale Clouds (AWS, GCP, Azure)

Hyper-scale cloud providers like Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure are industry standards for enterprise-grade applications; they were thoroughly evaluated and rejected for this project. The decision to use platforms is rooted in a direct conflict between their structural limitations and the project's non-functional requirements (NFRs).

As a team, we identified three core dimensions where hyper-scalers failed to meet project constraints:

Financial Volatility

- **The Constraint:** The project works under a strict R0/month budget threshold.
- **The Hyper-Scaler Failure:** AWS, GCP, and Azure operate on utility-based, pay-as-you-go pricing models. While they offer initial "Free Tiers," these tiers include hidden variable costs, mainly **Data Egress Fees** (charges for data leaving the cloud network).
- **Concurrency (NFR-02) Impact:** Under a stress-test scenario, simulating **500 concurrent users** continuously sending code payloads, streaming stdout/stderr buffers, and querying databases, the network traffic would quickly breach free data transfer caps. A minor configuration oversight or a malicious script could trigger automated scaling mechanisms, resulting in catastrophic, unbudgeted financial

liabilities **("bill shock")**. The selected stack (Vercel, Koyeb, Supabase, Oracle) hard-caps resource usage at the free limit, guaranteeing zero financial risk.

Severe Resource Starvation on Free Tiers

- **The Constraint:** The system must compile and execute multi-language user source code securely and return execution states in under 2 seconds.
- **The Hyper-Scaler Failure:** The compute instances provided under hyper-scaler free-tiers - such as the **AWS t2.micro/t3.micro** or **GCP e2-micro** – are very restricted. These machines provision only **1 vCPU and 1GB of RAM**, operating on a "burstable" CPU credit model.
- **Performance (NFR-01) & Security (NFR-03) Impact:** Code compilation (mainly for typed languages like Java or C++) is intensely CPU-bound. Forcing an application backend and a containerized execution engine onto a **1GB RAM** instance would cause **immediate kernel Out-of-Memory (OOM)** crashes under concurrent load. In addition, once burst credits are exhausted, the CPU is heavily throttled, destroying the target **< 2 second response time**. In contrast, Oracle Cloud's Always-Free tier grants access to 4 dedicated ARM vCPUs and 24GB of RAM, providing the physical hardware required to run Piston smoothly at scale.

Excessive Operational Complexity and Overhead

- **The Constraint:** The deployment must be highly maintainable, fully automated, and reproducible under a tight 2-hour implementation window.
- **The Hyper-Scaler Failure:** Architecting a containerized technical assessment platform on AWS requires manual configuration or heavy scripting for Virtual Private Clouds (VPCs), Identity and Access Management (IAM) security policies, Subnets, Internet Gateways, and Elastic Container Service (ECS) cluster definitions.
- **Maintainability (NFR-05) Impact:** This "Click-Ops" or extensive Infrastructure-as-Code (IaC) overhead introduces significant configuration friction. For a small development team, managing cloud-provider-specific network security patches takes time away from product feature development. Platforms like Vercel and Koyeb abstract this infrastructure layer entirely through native GitOps pipelines, allowing the team to meet the **2-hour automated deployment window with** zero infrastructure configuration maintenance.